

01 - Introductie

📅 09-09-2026 ⌚ Ongeveer 23 minuten 🗨️ Ongeveer 3426 woorden

Welkom bij **Programming Advanced!** 🙌 In de voorgaande trajecten heb je de fundamenteën van het .NET-ecosysteem en de C#-taal onder de knie gekregen. Je bent inmiddels vertrouwd met essentiële bouwstenen zoals datatypes, lussen en collecties, en je weet hoe je logica structureert en hergebruikt in methodes. Daarnaast heb je geleerd hoe je applicaties laat communiceren met de buitenwereld door data te lezen uit en te schrijven naar externe bestanden. Tot slot heb je een eerste kennismaking gehad met de fundamenteën van objectgeoriënteerd programmeren.

In dit onderdeel schakelen we een versnelling hoger. We verlaten de basis en gaan dieper in op de architectuur van professionele software. De focus verschuift van het schrijven van losse functionaliteiten naar het ontwerpen van robuuste systemen.

API - De motor op de achtergrond: 100% Backend

In deze module focussen we ons volledig op de `Backend` door gaandeweg te leren hoe je een eigen `Application Programming Interface (API)` ontwerpt en implementeert. Hoewel het bouwen van een API zonder visuele 'franjes' soms abstract of zelfs wat 'saai' kan aanvoelen, is dit waar de echte magie gebeurt. Beschouw de backend als de motor van een auto: zonder motor heb je niets aan een glimmend koetswerk. Het grote voordeel van een sterke API? Zodra deze staat, is hij universeel. Of je nu later een website, een krachtige desktop-app of een vlotte mobiele applicatie wilt maken: ze kunnen allemaal op dezelfde API worden aangesloten. Je bouwt hier dus de fundering voor elk denkbaar platform.

Wat is een API? (En waarom moet je dit willen bouwen?)

Een api is een soort verbindingsbrug tussen twee softwarecomponenten. De API geeft toegang tot functionaliteiten zonder dat een programma alle achterliggende details hoeft te weten. Op die manier kun je als bedrijf bijvoorbeeld gegevens delen met andere bedrijven. Bekende voorbeelden van zulke gegevens zijn weersgegevens, afbeeldingen, video's of locatiegegevens. Dat zijn gigantische hoeveelheden data die op verschillende manieren kunnen worden geïnterpreteerd of geanalyseerd. In plaats van zelf software te ontwikkelen voor al die verschillende mogelijkheden, maakt een bedrijf een API (interface) zodat andere bedrijven aan die data kunnen.

Welkom bij restaurant 'Chez le PC Cassée'

Stel je voor: je zit in een restaurant. Jij bent de **bezoeker** aan een tafeltje. Achterin het restaurant bevindt zich de **keuken**, waar de koks allemaal heerlijke ingrediënten (**data**) hebben liggen.

Je kan en mag niet zomaar zelf de keuken inrennen om in de potten te roeren. Dat zou een enorme chaos worden, en het is bovendien niet veilig. Je hebt dus een tussenpersoon nodig: de **ober**, en een overzicht van wat er mogelijk is: de **menukaart**.

- Jij bekijkt de menukaart en geeft een bestelling door aan de ober.
- De ober loopt naar de keuken en zegt: "*Tafel 4 wil graag de lasagne.*"
- De keuken maakt het eten klaar en geeft het bord aan de ober.
- De ober komt terug naar jouw tafel en serveert de lasagne.



De vertaalslag naar software

In de programmeerwereld werkt dit exact hetzelfde. Vervang de onderdelen uit het restaurant nu door software-termen:

- **De bezoeker (Jij) = De client:** Dit is de app op je smartphone, een website in je browser, of een slimme koelkast.

- **De keuken = De database / server:** Dit is de plek waar alle data veilig opgeslagen staat (bijvoorbeeld alle gebruikersnamen, producten of het actuele weer).
- **De ober & menukaart = De API:** Dit is de softwarelaag die ertussen zit. De API heeft een vaste lijst met regels (de menukaart): *"Als je mij een geldige gebruikersnaam stuurt via een 'GET-verzoek', dan loop ik naar de database en kom ik terug met het profiel van die gebruiker."*

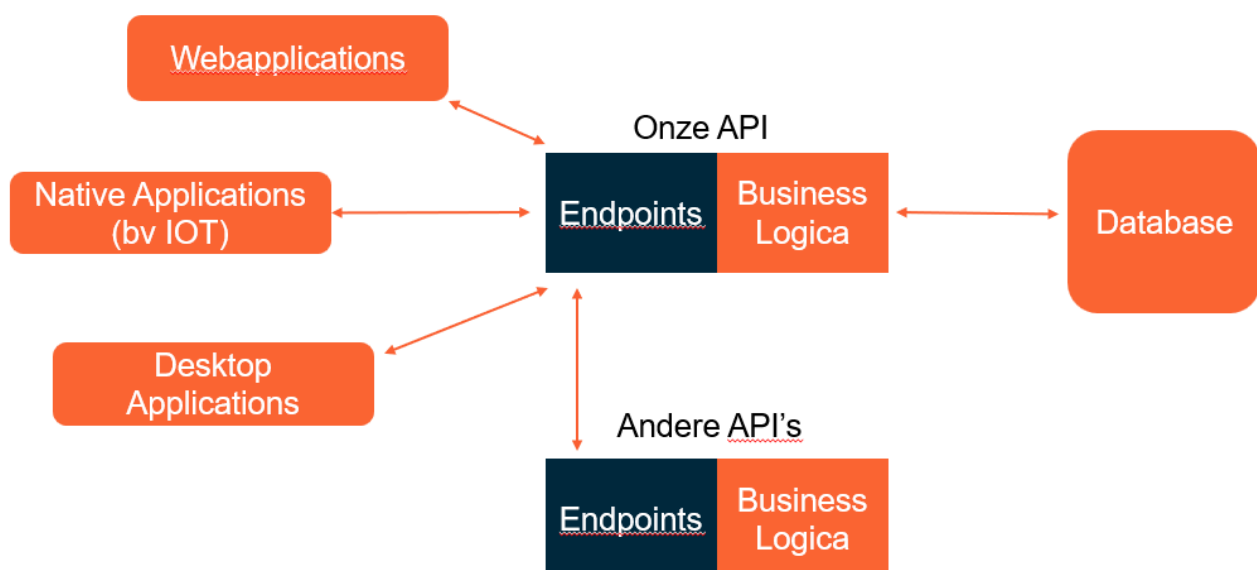
Info

💡 **Kortom:** Een API (*Application Programming Interface*) is de **digitale ober** die ervoor zorgt dat verschillende systemen veilig, snel en gestructureerd met elkaar kunnen praten!

Waarom is dit de "Holy Grail" voor developers?

Als software engineer bouw je een API om drie hele belangrijke redenen:

1. **Bouw één keer, gebruik overal:** Je programmeert de logica (berekeningen, database-koppelingen en beveiliging) slechts één keer in je API. Daarna kun je daar supermakkelijk een website, een iPhone-app én een Android-app tegelijkertijd op aansluiten. Ze praten namelijk allemaal met diezelfde 'ober'.
2. **Veiligheid (De Uitsmijter):** De API zorgt ervoor dat de buitenwereld nooit rechtstreeks in je database kan rommelen. De API controleert eerst of je wel mag bestellen: *"Heb je de juiste digitale sleutel (token) bij je? Nee? Dan krijg je deze data niet te zien."*
3. **Heavy Lifting:** Je smartphone hoeft zelf geen gigantische berekeningen uit te voeren. Dat doet de krachtige server waar de API op draait. De app op je telefoon hoeft alleen maar het resultaat te tonen.



Wat ga je leren in deze Module?

In deze module transformeer je van een programmeur die "code schrijft die werkt" naar een software engineer die systemen ontwerpt. We duiken in de volgende kernthema's:

- **API Design & Ontwikkeling:** Je leert hoe je een professionele API schrijft, gebruikt en test vanaf nul. Je ontwerpt logische eindpunten en leert hoe je jouw backend toegankelijk maakt voor de buitenwereld.
- **Architectuur & SOLID-principes:** Schrijf code die niet bezwijkt onder zijn eigen complexiteit, maar flexibel en herbruikbaar blijft.
- **Interfaces & Abstractie:** Leer hoe je componenten loskoppelt zodat je onderdelen van je systeem makkelijk kunt vervangen of uitbreiden.
- **Data-opslag & Databases:** Ontwerp complexe datamodellen of koppel je applicatie aan een database voor permanente opslag.
- **Modellen & DTO's:** Leer het cruciale verschil tussen interne datastructuren en de objecten waarmee je met de buitenwereld communiceert.
- **Security Essentials:** Beveilig je API en zorg dat gevoelige data enkel toegankelijk is voor de juiste gebruikers.
- **Automatische Testing:** Garandeer de betrouwbaarheid en toekomstige werking van je systeem met geautomatiseerde Unit Tests.

Jouw Toolbelt

Om deze backend-wereld tastbaar te maken, maken we gebruik van industrie-standaard tools:

- **OpenAPI & Scalar:** We gebruiken de ingebouwde .NET OpenAPI-engine om de structuur van onze API te beschrijven. Met Scalar genereren we automatisch een mooie, interactieve documentatiepagina zodat je direct kunt zien en testen wat je gebouwd hebt.
- **C# en .Net 10:** De krachtige Object-Georiënteerde programmeertaal waarmee we al wat ervaring hebben.
- **Postman:** De 'browser voor developers'. Hiermee leer je hoe je complexe verzoeken naar API's stuurt en de antwoorden analyseert.
- **Data Access Duo (EF Core & Dapper):** Je leert werken met twee verschillende benaderingen van data-opslag.
 - **Entity Framework Core:** De krachtige ORM-brug tussen je C#-code en de database die de queries abstraheert. Met 1 EF Query kan je vrijwel alle grote database

technologieën aanspreken.

- **Dapper:** Een 'micro-ORM' voor maximale performance en volledige controle over SQL-queries.
- **Docker:** We gebruiken containers om onze ontwikkelomgeving (zoals databases) razendsnel op te zetten en te garanderen dat de applicatie overal identiek draait.
- **PostGres SQL:** De professionele database-omgeving waarin we onze data daadwerkelijk gaan opslaan en beheren.

REST: De standaard van het internet

Hoewel er verschillende smaken API's bestaan (zoals GraphQL of SOAP), focussen wij ons op de absolute koning van het internet: de **REST API**.

Een REST API spreekt de taal van het internet: **HTTP**. Waar je eerder in JavaScript (`fetch`) of bij IT-Challenges al data hebt *opgevraagd* van andermans API's, ga je nu leren hoe je zelf die "ober" programmeert.

Je leert werken met de vier belangrijkste acties (de zogenaamde `CRUD-operaties`):

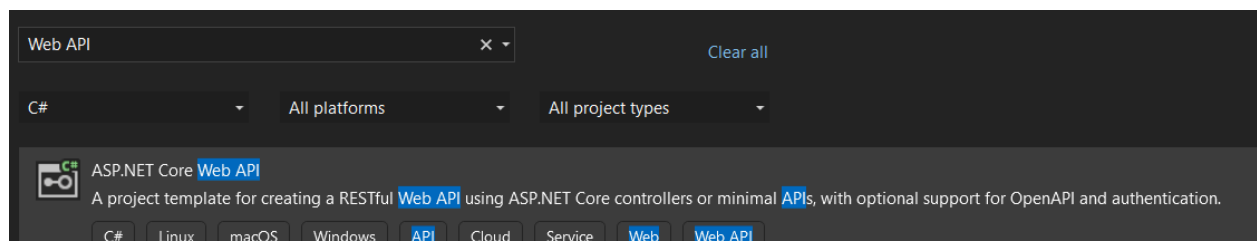
- **GET:** "Geef me de gegevens." (Read)
- **POST:** "Maak dit nieuwe item aan." (Create)
- **PUT:** "Werk dit bestaande item bij." (Update)
- **DELETE:** "Gooi dit weg." (Delete)

Het resultaat? Een razendsnelle verbinding die data meestal teruggeeft in **JSON**-formaat: een simpel tekstformaat dat elke programmeertaal ter wereld begrijpt.

Je eigen API aanmaken

Visual Studio

Maak in Visual Studio een nieuw project **ASP.NET Core Web API** aan met als naam `WebAPIDemo` . Kies als Framework **.NET 10 (Long Term Support)**.



Additional information

ASP.NET Core Web API C# Linux macOS Windows API Cloud Service Web Web API

Framework ⓘ
.NET 10.0 (Long Term Support)

Authentication type ⓘ
None

☒ Configure for HTTPS ⓘ
☐ Enable container support ⓘ

Container OS ⓘ
Linux

Container build type ⓘ
Dockerfile

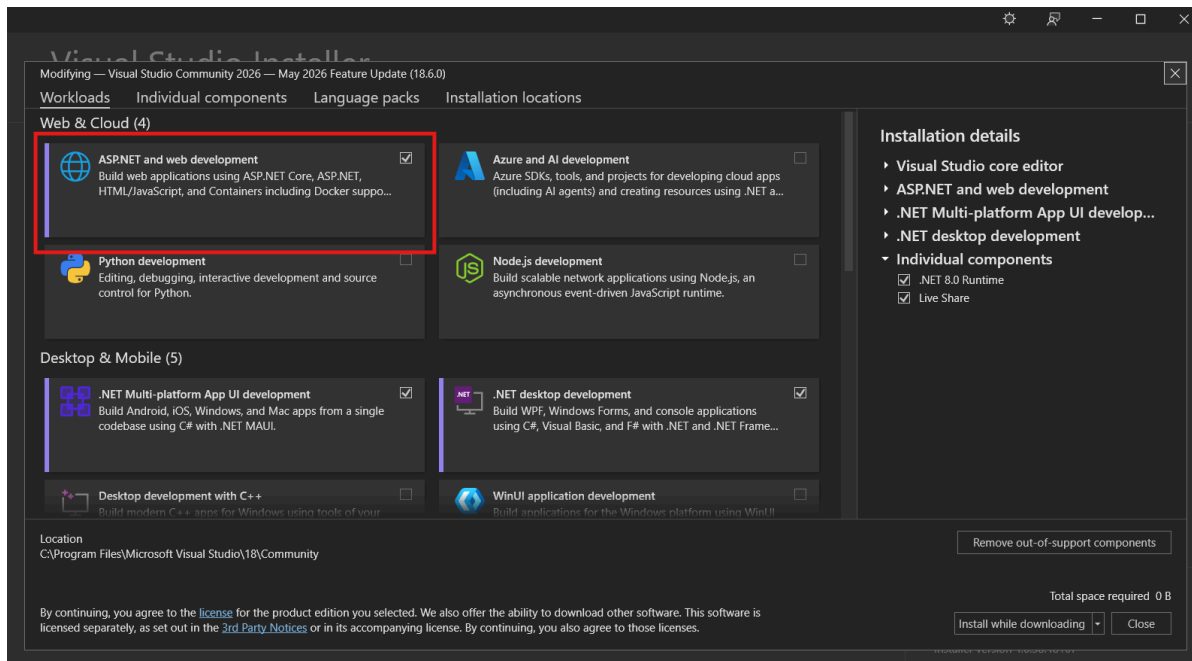
☒ Enable OpenAPI support ⓘ
☐ Do not use top-level statements ⓘ
☒ Use controllers ⓘ
☐ Enlist in Aspire orchestration ⓘ

May 2026 Feature Update

Back Create

Notitie

Als je deze optie niet ziet zal je de correcte workload moeten installeren. Open je Visual Studio Installer en controleer of de **ASP.NET and Web development** workload geïnstalleerd staat.



Terminal

Via de terminal kun je ook een nieuw Web API project aanmaken. Visual Studio maakt standaard een Solution aan met een nieuw project. In VSC kun je dus best ook eerst een solution maken:

```
1 # nieuw project aanmaken                                pwsh
2 dotnet new sln -n WebAPIDemo
3
4 # nieuw .NET 10.0 project aanmaken dat gebruik maakt van Controllers
5 dotnet new webapi -n WebAPIDemo --framework net10.0 --use-controllers
6
7 # project toewijzen aan de solution
8 dotnet sln WebAPIDemo.sln add WebAPIDemo/WebAPIDemo.csproj
```

Demoproject

Een nieuw project bevat een demo van een Weer-API. Hierin zijn volgende belangrijke onderdelen aanwezig:

- **Program.cs:** Dit is de motor en het startpunt van de applicatie. Hier stel je in welke services (zoals databases of security) de app gebruikt en hoe de webserver zich gedraagt. We houden deze klasse liefst zo clean mogelijk; we voegen hier geen business logica toe.
- **Controllers\WeatherForecastController.cs:** Dit is een voorbeeld van een Controller . Een klasse die zal fungeren als het *brein van je applicatie*.
- **WeatherForeCast.cs:** Dit is een voorbeeld van een Model . Een klasse die data bevat die doorheen de applicatie getransporteerd wordt en vaak *een codevoorstelling is van een echt object of concept* zoals een klant, een verkoop, een product etc.
- **appsettings.json:** Het configuratiebestand voor instellingen die per omgeving kunnen verschillen, zoals je database-connectiestring.
- **properties\launchSettings.json:** Hier vind je terug op welke URL's en poorten de web-server lokaal opstart.

Controller(s): Het brein van je API

Info

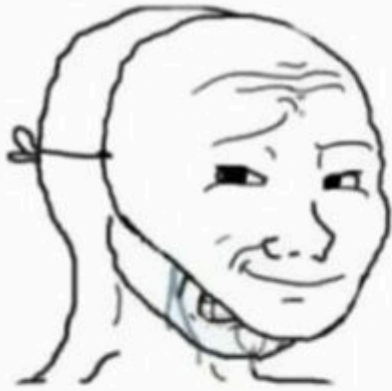
Een Controller klasse is het hart van je API-logica. Waar je in Programming Essentials bijna alles in Program.cs schreef, verhuist die business code nu naar Controllers.

Een Controller is verantwoordelijk voor:

- Ontvangen van een verzoek (bijv. een GET-request).
- Verwerken van de logica (berekeningen, data ophalen).
- Terugsturen van een antwoord naar de gebruiker.

Documentatie & Testen activeren met Scalar

In .NET 10 levert Microsoft de ruwe OpenAPI-data aan, maar zit er standaard geen visuele interface meer ingebouwd. In .Net 8 zat `Swagger`, een visueel framework om endpoints weer te geven en te testen via de browser, maar deze heeft Microsoft met pensioen laten gaan. Om onze API makkelijk te testen vanuit de browser, installeren we de moderne industriestandaard `Scalar`. Als developers willen we *het wiel niet heruitvinden*. Vaak bestaat er al kant-en-klare functionaliteit die we in ons project kunnen gebruiken zonder het zelf te moeten schrijven. Deze functionaliteit staat bekend als `packages` en kunnen we installeren via de `Nuget Package Manager` of de `Terminal`.

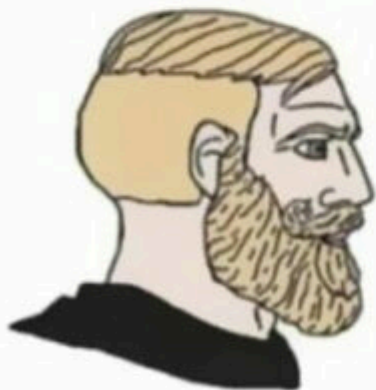


Look, we have similar ideas.

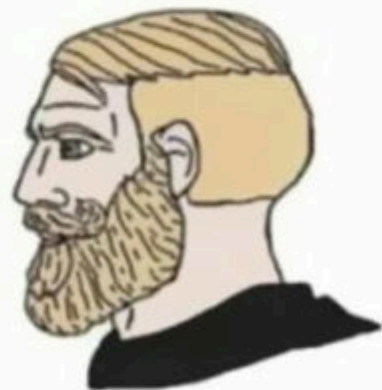


No! You stole my idea.

PROGRAMMERS



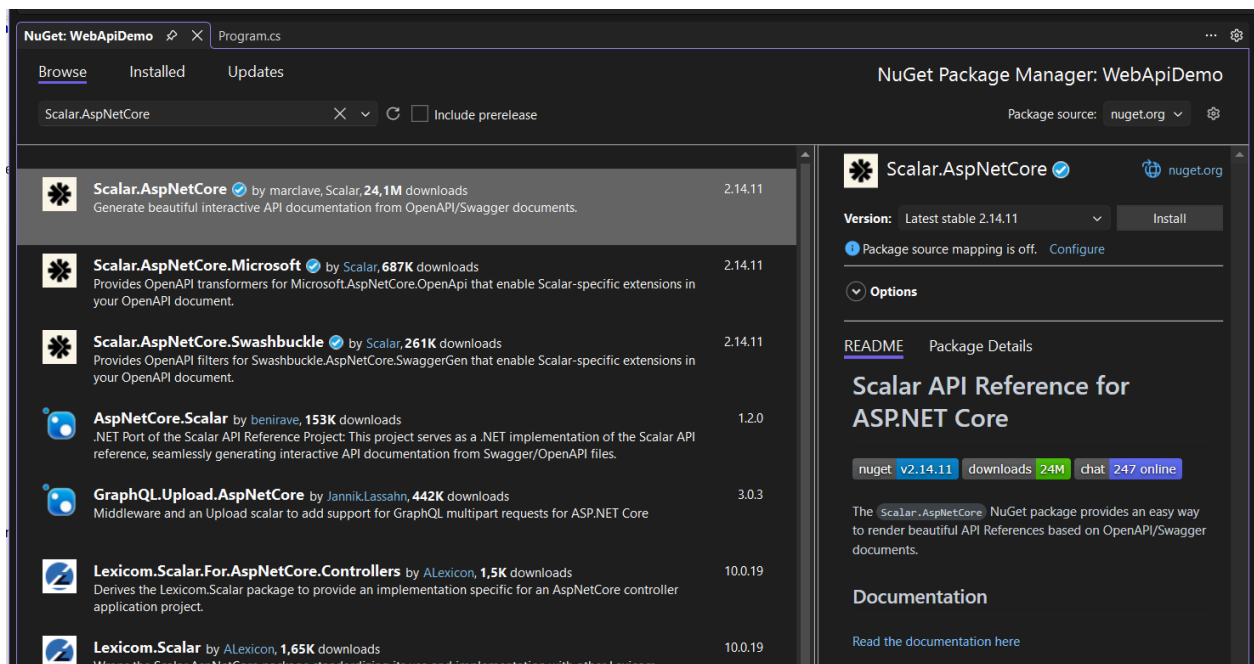
Man, I stole your code.



It's not my code.

Nuget Package Manager

Rechtsklik op je project in de Solution Explorer en klik op **Manage Nuget Packages**. Klik daarna op de **Browse** tab en installeer de **Scalar** package



Terminal

Open de terminal in de map van je project en voer het volgende commando uit:

1

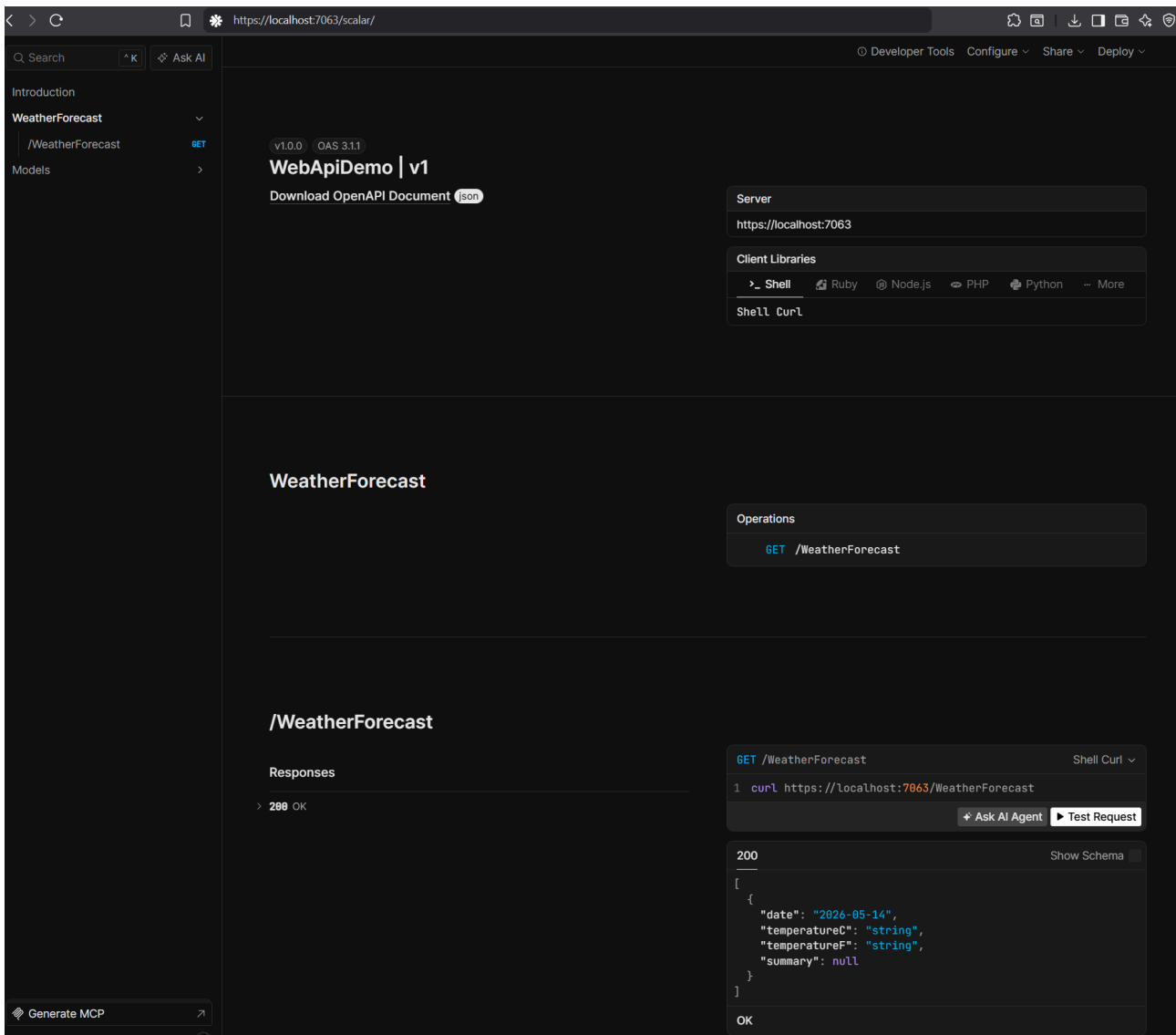
```
dotnet add package Scalar.AspNetCore
```

pwsh

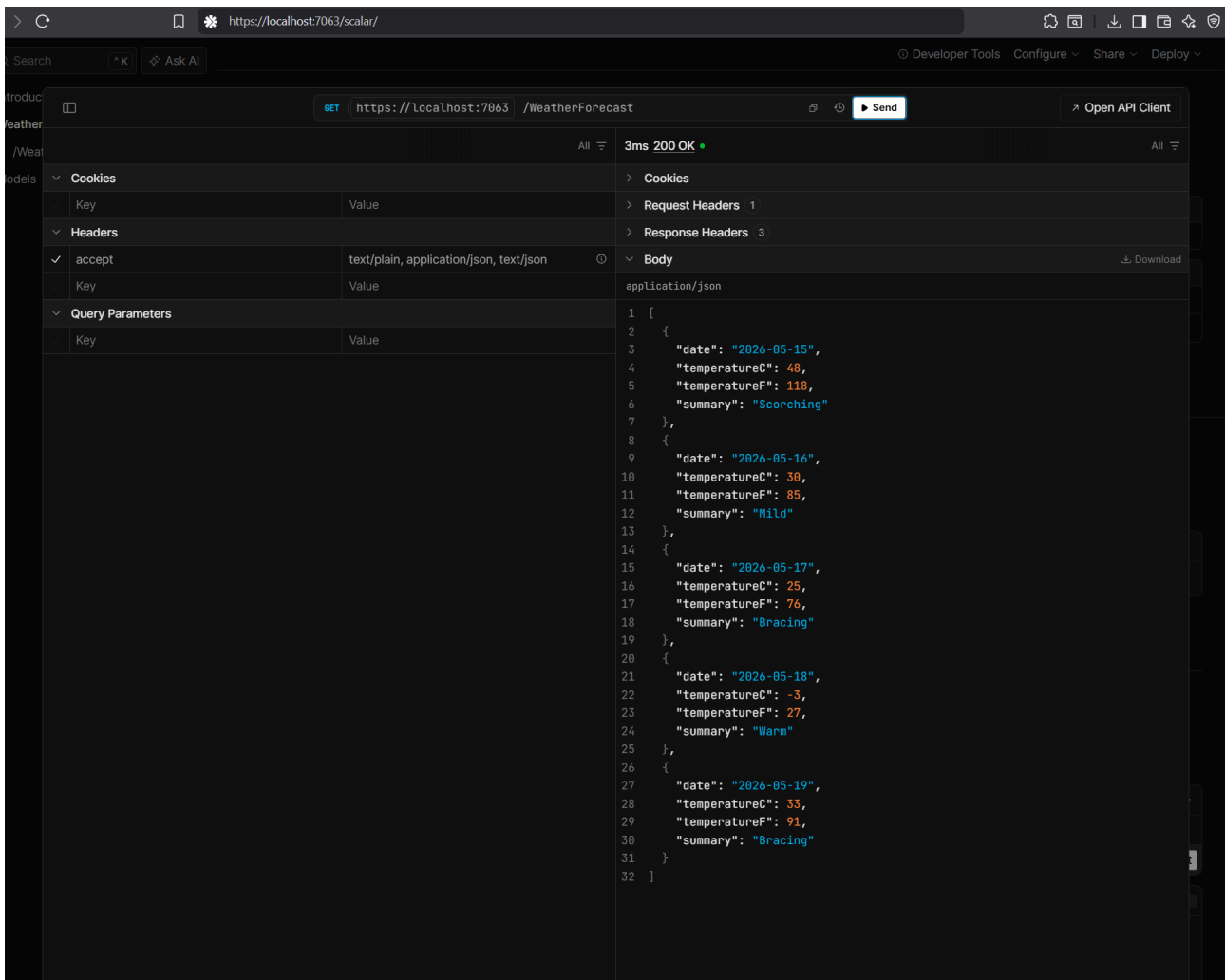
Open vervolgens **Program.cs**. Zorg ervoor dat `app.MapScalarApiReference()` wordt aangeroepen binnen de `IsDevelopment()` controle, vlak onder `app.MapOpenApi()`:

```
1  if (app.Environment.IsDevelopment())                                csharp
2  {
3      app.MapOpenApi();
4      app.MapScalarApiReference(); // <-- Voeg dit toe voor de interactieve
5  UI
   }
```

Als je het project start, zul je een command-venster terugvinden in je taakbalk. Achterliggend is een webserver opgestart die luistert naar inkomende verzoeken. C# kan niet rechtstreeks gecompileerd worden door een browser. Daarom heb je aan de serverkant een webserver nodig die de backend-logica uitvoert. In het command-venster kun je informatie van deze webserver terugvinden. Je kan `ctrl+klik` uitvoeren op de gegenereerde link in dit window om je API in de browser te openen. Voeg daarna `/scalar` achter je lokale URL toe (bijv. `https://localhost:xxxx/scalar`). Dit is de interactieve documentatiepagina van je API.

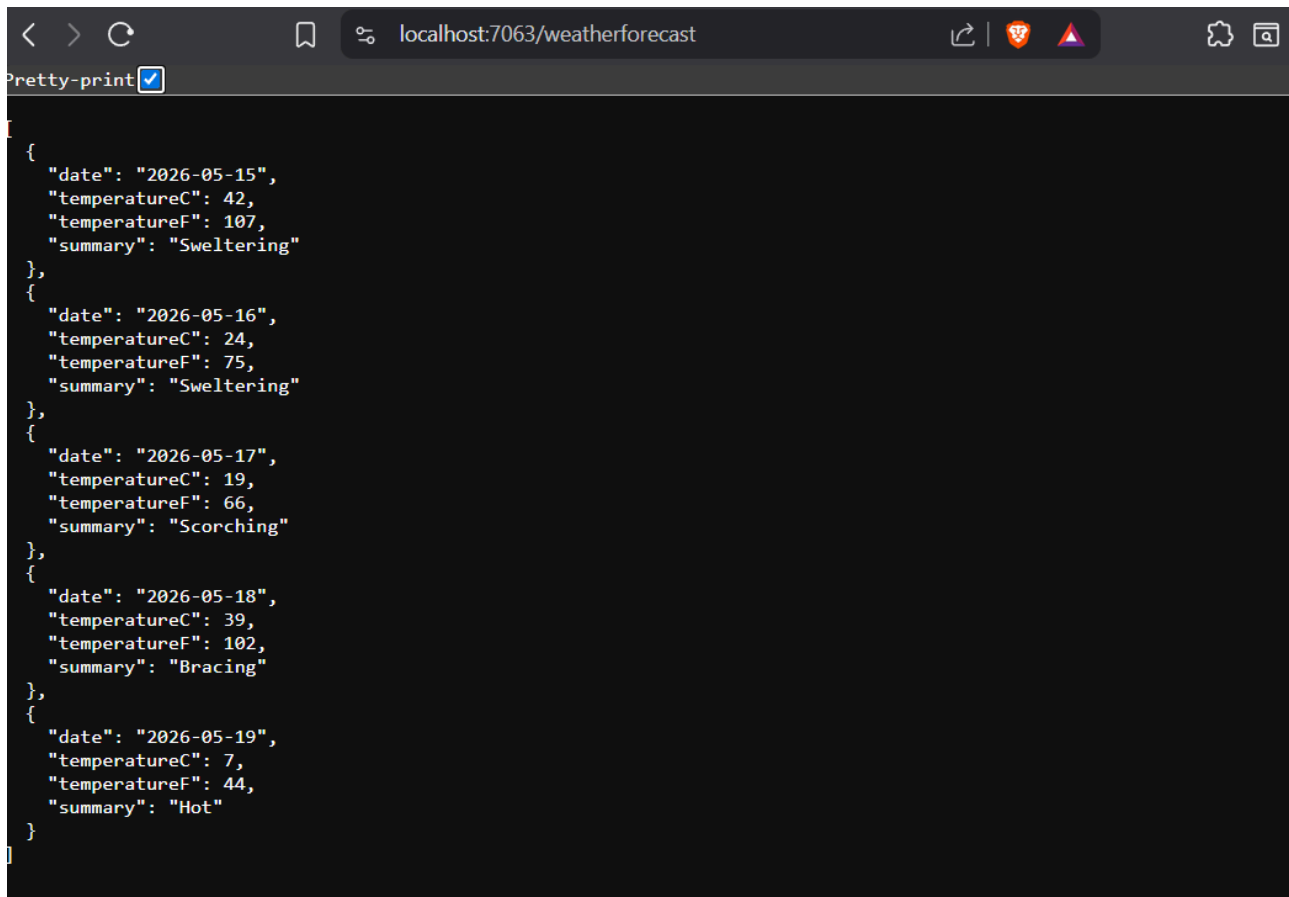


- Klik aan de linkerkant op het GET /weatherforecast endpoint.
- Klik rechts op de knop Test Request of Test.
- Klik op Send.
- Je krijgt een HTTP-statuscode 200 OK (succes) en de resultaten overzichtelijk gepresenteerd in JSON-formaat.



Je kunt ook rechtstreeks naar het endpoint surfen via

`https://localhost:xxxx/weather-forecast`. Het resultaat van de pagina is nu enkel een ruwe lijst van gegevens in JSON-formaat.



```
[
  {
    "date": "2026-05-15",
    "temperatureC": 42,
    "temperatureF": 107,
    "summary": "Sweltering"
  },
  {
    "date": "2026-05-16",
    "temperatureC": 24,
    "temperatureF": 75,
    "summary": "Sweltering"
  },
  {
    "date": "2026-05-17",
    "temperatureC": 19,
    "temperatureF": 66,
    "summary": "Scorching"
  },
  {
    "date": "2026-05-18",
    "temperatureC": 39,
    "temperatureF": 102,
    "summary": "Bracing"
  },
  {
    "date": "2026-05-19",
    "temperatureC": 7,
    "temperatureF": 44,
    "summary": "Hot"
  }
]
```

Routing: Hoe wordt je code gevonden?

In een Console-app startte je code simpelweg bij de eerste regel. In een API bepaal je via Routing welke code uitgevoerd moet worden bij welke URL. *Met Routing bedoelen we dat we een specifieke actie koppelen aan het navigeren naar een bepaalde HTTP Route.*

Het is je mogelijk al opgevallen dat we een Controller genaamd WeatherForecastController hebben en er een endpoint genaamd weatherforecast beschikbaar is. Dit is geen toeval. API's gebruiken de namen van een Controller klasse zonder de *-Controller* suffix om endpoints aan te maken. Standaard wordt er minstens 1 GET actie voorzien.

In de WeatherForecastController zie je bovenaan `[Route("[controller]")]`. Dit zorgt ervoor dat als je surft naar `https://localhost:xxxx/weatherforecast`, deze specifieke klasse wordt aangesproken. Met Attributes zoals `[HttpGet]` bepaal je vervolgens welke methode binnen die klasse reageert op de actie van de gebruiker.

Je eigen endpoint aanmaken

Voeg aan de Controller folder een nieuwe klasse toe en noem deze WelkomController.

Visual Studio

- Klik met de rechtermuisknop op de map Controllers -> Add New Item -> **API Controller - Empty**
- Geef de controller de naam **WelkomController.cs** en klik op Add.

Terminal / VS Code

Maak handmatig een nieuw bestand aan in de map Controllers en noem het **WelkomController.cs**.

Zorg ervoor dat je klasse overerft van `ControllerBase` (en niet van `Controller`), aangezien we een pure backend API bouwen zonder visuele HTML-views. Vul de klasse aan met de volgende code:

```
1  using Microsoft.AspNetCore.Mvc;                                     csharp
2
3  namespace WebAPIDemo.Controllers;
4
5  [ApiController]
6  [Route("api/[controller]")]
7  public class WelkomController : ControllerBase
8  {
9      // Dit endpoint reageert op: GET /welkom
10     [HttpGet]
11     public string GetWelkomBericht()
12     {
13         return "Welkom bij Programming Advanced! Je eerste eigen endpoint
14 werkt 🚀";
15     }
16
17     // Dit endpoint reageert op: GET /welkom/gepersonaliseerd/Anna
18     [HttpGet("gepersonaliseerd/{naam}")]
19     public string GetGepersonaliseerdBericht(string naam)
20     {
21         return $"Welkom bij Programming Advanced, {naam}! 🚀";
22     }
23 }
```

Hoe is deze Controller opgebouwd?

- **[ApiController]:** Dit attribuut vertelt het ASP.NET Core framework dat deze klasse een API-controller is. Dit zorgt onder andere voor automatische modelvalidatie, betere foutmeldingen en dat deze klasse aangesproken kan worden door Http Requests.
- **ControllerBase:** Onze klasse erft over van ControllerBase. Dit is de standaard, lichtgewicht basisklasse voor API-controllers. Het geeft ons toegang tot ingebouwde HTTP-responses (zoals Ok(), NotFound(), Created()).
- **[Route("[controller]")]:** Het framework vervangt [controller] automatisch door de naam van de klasse zonder het achtervoegsel 'Controller'. Merk de api/ prefix voor [controller] op. Het is een best practice om api rechtstreeks in de url te benoemen. Jouw basisroute wordt hierdoor dus localhost:xxxx/api/welkom
- **Routeparameters:** In de tweede methode gebruiken we [HttpGet("gepersonaliseerd/{naam}")]. Hiermee breiden we de route uit. Wat de gebruiker op de plaats van {naam} invult in de URL, wordt door het framework automatisch gekoppeld aan de inputparameter string naam van de C#-methode.

Wanneer je de applicatie nu opnieuw opstart en naar /scalar navigeert, zul je zien dat je nieuwe Welkom-endpoints automatisch aan de documentatie zijn toegevoegd en klaar zijn om getest te worden! Merk op dat je aan Gepersonaliseerd nu ook input parameter(s) kan toevoegen!

The screenshot shows the Scalar API Reference interface. The browser address bar displays `localhost:7063/scalar/#tag/welkom/GET/api/Welkom`. The left sidebar contains a search bar and a navigation menu with items: Introduction, WeatherForecast, Welkom (expanded), /api/Welkom (selected), /api/Welkom/gepersonaliseerd/{naam}, and Models. The main content area is titled 'Welkom' and includes an 'Operations' section with two GET endpoints: `/api/Welkom` and `/api/Welkom/gepersonaliseerd/{naam}`. Below this, the selected endpoint `/api/Welkom` is detailed, showing a '200 OK' response. A 'Test Request' section provides a curl command: `curl https://localhost:7063/api/Welkom`. The response details show a status of 200, a string type, and the text 'OK'. The bottom endpoint `/api/Welkom/gepersonaliseerd/{naam}` is also visible, showing a '200 OK' response. A 'Generate MCP' button is located at the bottom left of the sidebar.

Info

Merk op dat enkel de namen van de Controllers ervoor zorgen dat een bepaald endpoint wordt aangemaakt. Binnenin eenzelfde endpoint moeten we gebruik maken van een `Route` en een `HTTPVerb` (GET, POST, DELETE, ...) om een bepaalde methode aan te spreken. De namen van de methodes spelen hierin geen rol, de gebruiker krijgt deze niet te zien. Deze zijn enkel voor ons programmeurs relevant om snel de intentie van een methode te begrijpen.

Alternatief: Postman - De absolute industriestandaard

Indien je liever niet via een browser werkt, en liever gebruikmaakt van een krachtige, dedicated testsuite die je commando's, query's en instellingen netjes onthoudt, biedt Postman een uitstekend alternatief (en voor grotere projecten zelfs een enorme verbetering) tegenover Scalar.



Postman is de go-to tool voor zowat alle professionele API-developers wereldwijd. Waar Scalar perfect is voor een snelle visuele test in de browser, blinkt Postman uit in het structureren en bewaren van je werk. Je kunt je verzoeken opslaan in overzichtelijke mappen

(zogenaamde Collections), werken met omgevingsvariabelen en zelfs complexe scenario's testen zonder je browser te hoeven openen.

Je kan Postman hier downloaden: <https://www.postman.com/downloads/>

Info

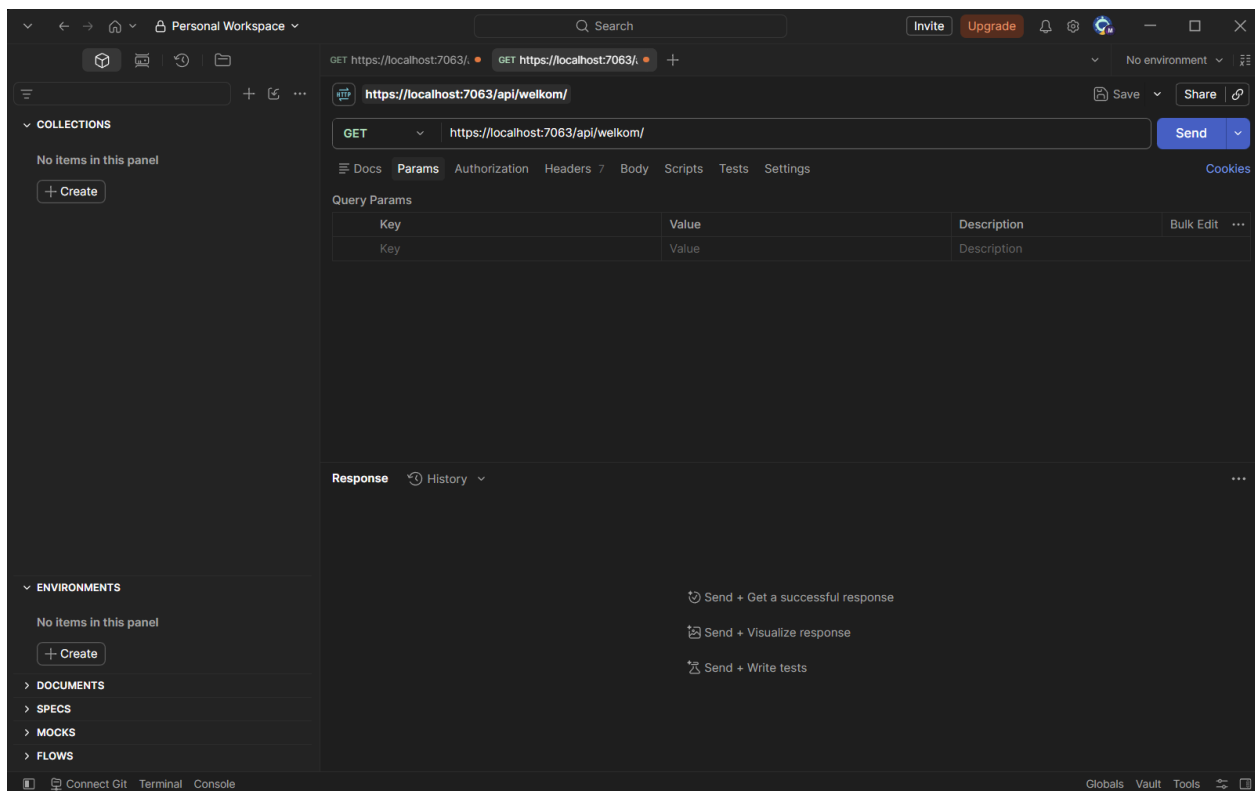
💡 Geen paniek: Om snel kleine API's te testen, zoals we in het begin van deze cursus doen, is Scalar meer dan voldoende. Voor de oefeningen en examens maakt het overigens niet uit welke tool je gebruikt. Postman is vooral handig voor complexe(re) queries die veel input nodig hebben of bepaalde authenticatie cookies nodig hebben. Probeer ze gerust allebei uit en gebruik degene die voor jou het meest intuïtief aanvoelt!

PostMan gebruiken

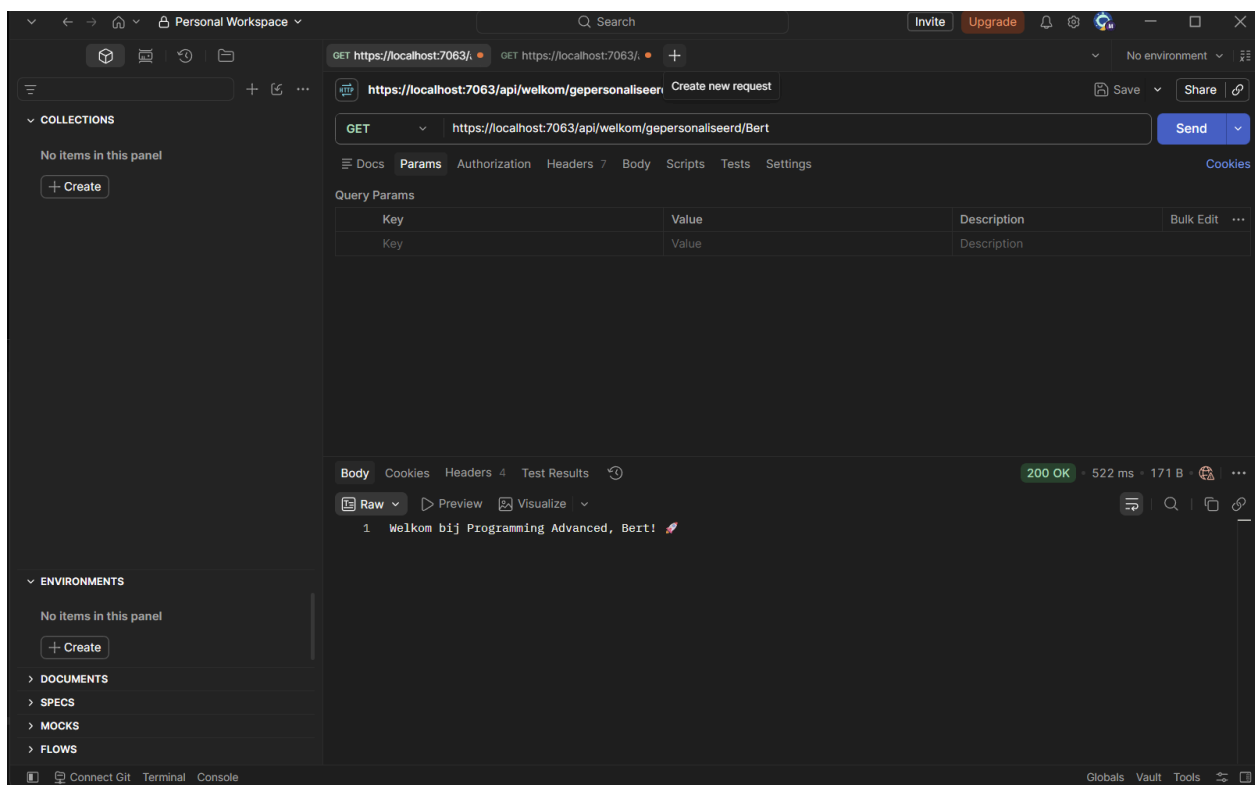
Zodra je de tool hebt geïnstalleerd en opgestart (je kunt een gratis account aanmaken om je veelgebruikte queries in de cloud op te slaan, maar je kunt die stap ook overslaan), gaan we direct de proef op de som nemen met ons nieuwe /welkom endpoint.

- Zorg ervoor dat je API lokaal draait (via Visual Studio of je Terminal).
- Klik in Postman op de knop New of het + icoontje bovenaan om een nieuwe tab, een HTTP Request, te openen.
- Aan de linkerkant van de adresbalk zie je een dropdown-menu dat standaard op GET staat. Dit is exact de HTTP Verb die we nodig hebben.
- Vul in de adresbalk de URL van je draaiende lokale API in. Vergeet niet de juiste poort en de routenaam toe te voegen, bijvoorbeeld: `https://localhost:7063/api/welkom/` (Vervang 7063 door het poortnummer dat in je opstart-console te zien is).
- Klik op de Send knop.

Standaard Get



Get met Input



Het resultaat analyseren

Kijk nu naar het onderste gedeelte van je scherm, de Response sectie. Als alles goed is gegaan, zie je hier twee belangrijke dingen:

- Aan de rechterkant zie je de statuscode `200 OK` in het groen oplichten, wat betekent dat de server je verzoek succesvol heeft verwerkt.
- In het Body venster zie je het daadwerkelijke antwoord van je Controller verschijnen: "Welkom bij Programming Advanced! Je eerste eigen endpoint werkt 🎉".

Test direct ook even je tweede endpoint door de URL aan te passen naar `https://localhost:xxxx/welkom/gepersonaliseerd/Anna`. Klik op Save (of Ctrl+S) om je request op te slaan in een Collection aan de linkerkant van je scherm. Zo hoeft je deze URL en parameters morgen niet opnieuw in te typen wanneer je verder werkt!

Samenvatting

Laten we eerlijk zijn: dit was een behoorlijk pittige kennismaking! 🤖 We hebben meteen met flink wat termen gegooid. Van abstracte concepten zoals *HTTP Verbs*, *Routing* en *REST-architectuur* tot gloednieuwe .NET 10 snufjes zoals *OpenAPI* en *Scalar*. Het is volkomen normaal als je hoofd nu een beetje tolt en de helft nog wat onwerkelijk aanvoelt. Onthoud: je bent aan het transformeren van een code-schrijver naar een software-engineer, dat vraagt nu eenmaal even wat klimwerk!

Ter herhaling:

- **Wat is een API?** De 'ober' die zorgt dat een app of website (frontend) veilig kan communiceren met de database (backend). Het doet al het zware reken- en beveiligingswerk op de server.
- **REST & HTTP** : De universele internettaal van API's. Data wordt uitgewisseld in JSON-formaat via vier hoofdacties (CRUD)
 - **GET**: Om data vlot op te vragen. (Read)
 - **POST**: Om gloednieuwe gegevens veilig aan te maken. (Create)
 - **PUT**: Iets bestaands bijwerken (Update)
 - **DELETE**: Om data die we niet meer nodig hebben resoluut weg te gooien. (Delete)
- **.NET 10** : Swagger (Swashbuckle) is definitief met pensioen. .NET 10 genereert nu zelf de ruwe data (OpenAPI). We installeren het pakket `Scalar` om hier via de URL `/scalar` een moderne, interactieve testpagina van te maken.

- **PostMan** : Alternatief kunnen we PostMan gebruiken om onze endpoints te testen. Postman is vooral handig om herhalende, complexe queries te testen.
- **Controllers** : Het brein van je API. Een controller-klasse vangt een verzoek op, voert de C#-logica uit en stuurt een antwoord terug.
- **ControllerBase** : API-controllers erven altijd over van ControllerBase. Dit is de vederlichte variant zonder de overbodige HTML-ballast die traditionele websites gebruiken.
- **Routing** : Via attributen boven je klassen en methoden bepaal je de URL. [Route("[controller]")] zorgt er bijvoorbeeld voor dat WelkomController luistert naar het webadres /welkom.
- **Routeparameters** : Door {variabele} in een route te plaatsen (bijv. [HttpGet("gepersonaliseerd/{naam}"))], pikt .NET de tekst uit de URL automatisch op als inputparameter voor je C#-methode.

Nu de theorie achter de rug is en de fundering staat, wordt het tijd voor het échte werk. In het volgende hoofdstuk gaan we de abstracte concepten omzetten in tastbare code. We zullen ons focussen op het maken van eenvoudige, functionele endpoints met de drie grote iconen van het internet: GET, POST en DELETE.

In het volgende hoofdstuk onze API écht tot leven wekken! 🚀

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM